NATURAL LANGUAGE PROCESSING

Word2Vec, CBOW & Skip-Gram

How computers learn word meaning from neighboring words

From intuition → architecture diagrams → real examples → a short Python demo

Numan Aslam

LEARNING GOALS

By the end of this lecture, you can…

1	**Explain embeddings**	Why a dense vector is more useful than a word ID or one-hot code
2	**Draw both models**	CBOW: context → center; Skip-Gram: center → context
3	**Build examples**	Generate training pairs using a context window
4	**Try the technique**	Train and inspect a small model with Python / Gensim

Why should a computer relate words?

REAL-WORLD SEARCH QUERY

“affordable notebook computer for university”



budget laptop

student computer

portable notebook

A useful language model can represent related words near one another.

Application example: semantic search in an e-commerce product catalog.

Problem: word IDs do not express meaning

doctor	`[1, 0, 0, 0]`
nurse	`[0, 1, 0, 0]`
pizza	`[0, 0, 1, 0]`
laptop	`[0, 0, 0, 1]`

What the computer sees

doctor ≠ nurse
No encoded similarity

What people know

doctor and nurse share a medical context.

A one-hot code is an identifier, not a learned semantic representation.

THE KEY INTUITION

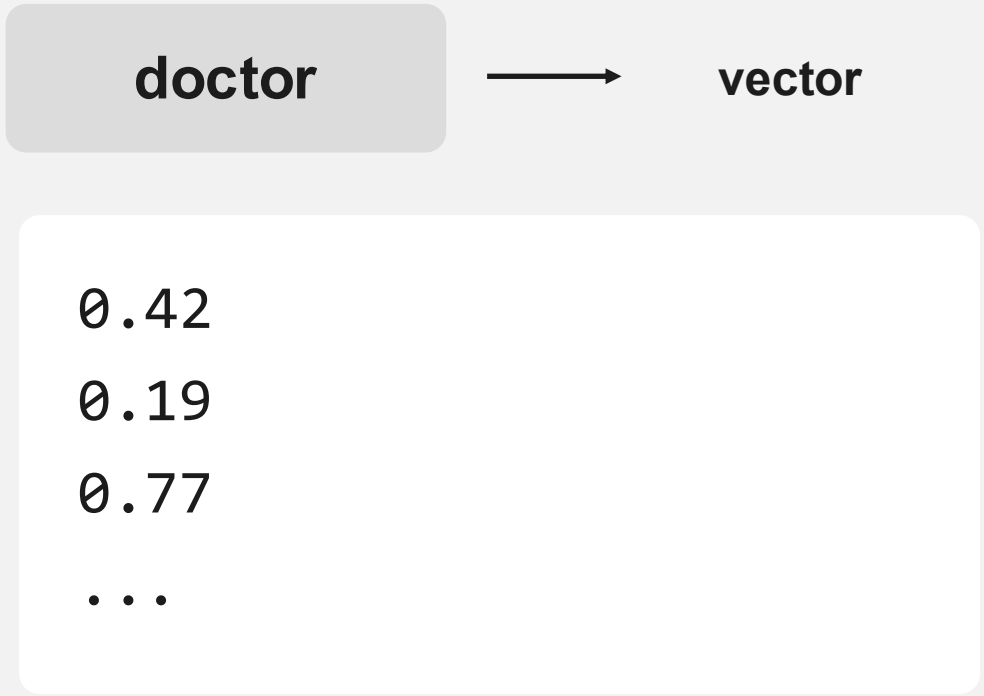
Words become similar through their contexts



doctor and nurse often appear near similar medical words

Toy sentences for intuition; similarity is learned from many examples, not two sentences alone.

A word becomes a learned numeric vector



Why vectors help

- Compare words numerically
- Find close neighbors
- Pass word features to downstream NLP models

Numbers are illustrative, not pretrained embedding values.

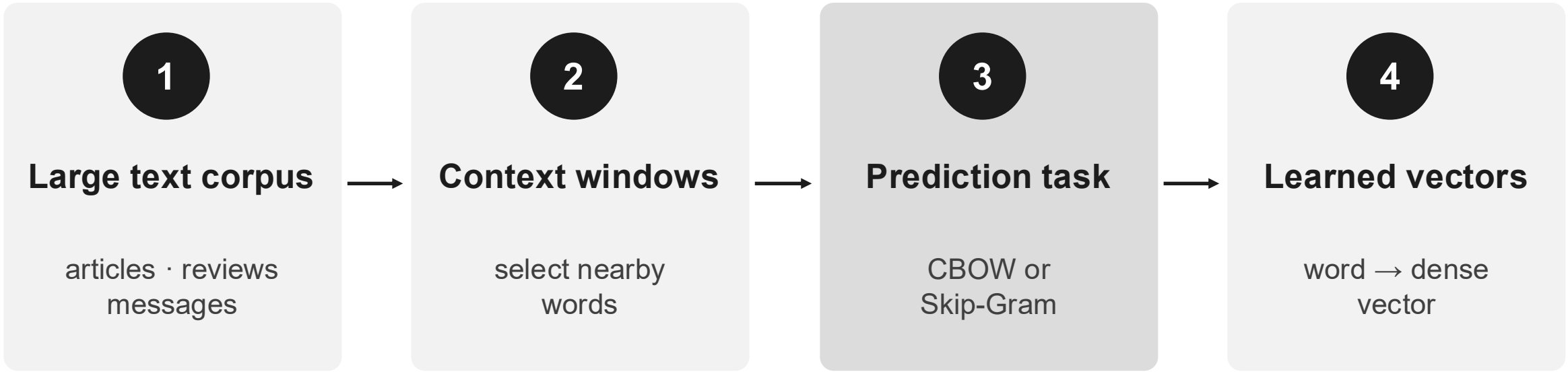
GEOMETRIC VIEW

Related words form neighborhoods



Conceptual positions only; this is not output from a trained model.

Learn vectors by solving a prediction task



We train a word prediction model to obtain reusable word vectors.

TRAINING DATA

A context window picks neighboring words

SENTENCE · WINDOW SIZE = 2

the	student	**drinks**	hot	tea	after	class
		CENTER WORD				

CONTEXT INPUT

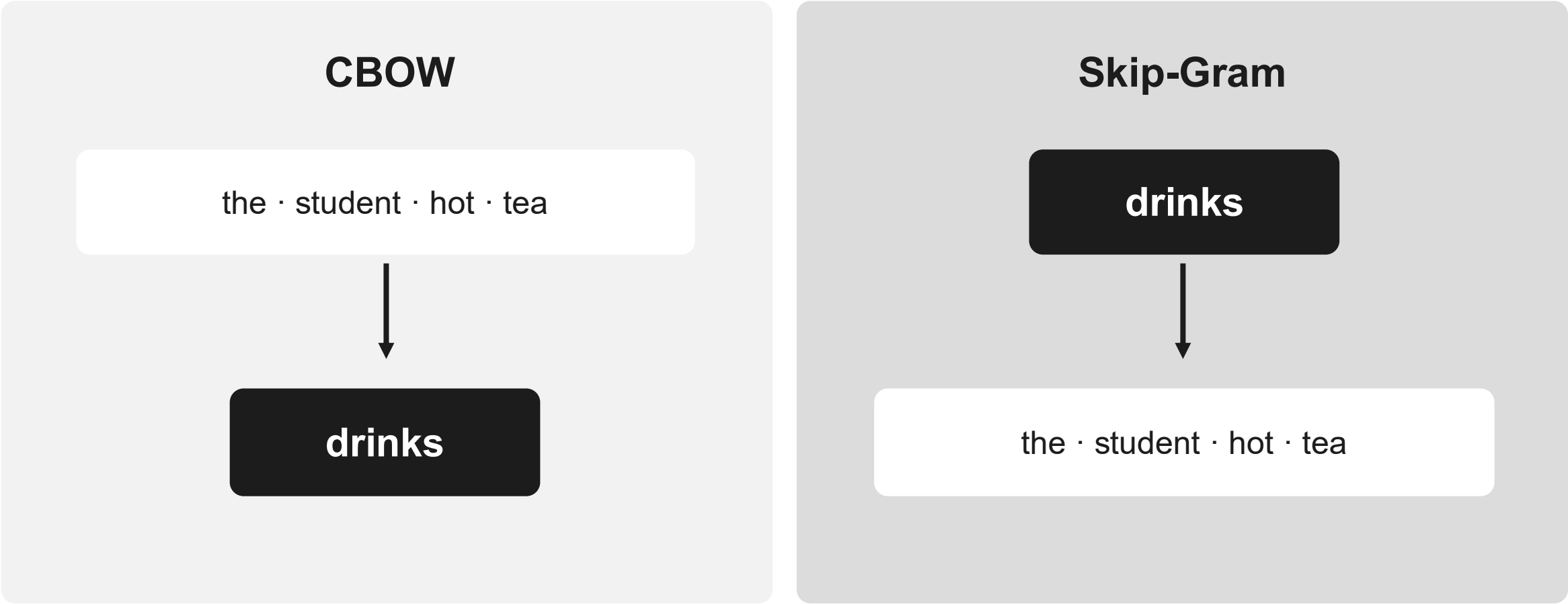
the · student · hot · tea

CENTER / TARGET

drinks

This same window supplies CBOW and Skip-Gram with opposite prediction directions.

Same words. Opposite prediction directions.



CBOW predicts one center word. Skip-Gram generates a prediction for each context word.

CBOW = guess the missing center word

FILL IN THE BLANK

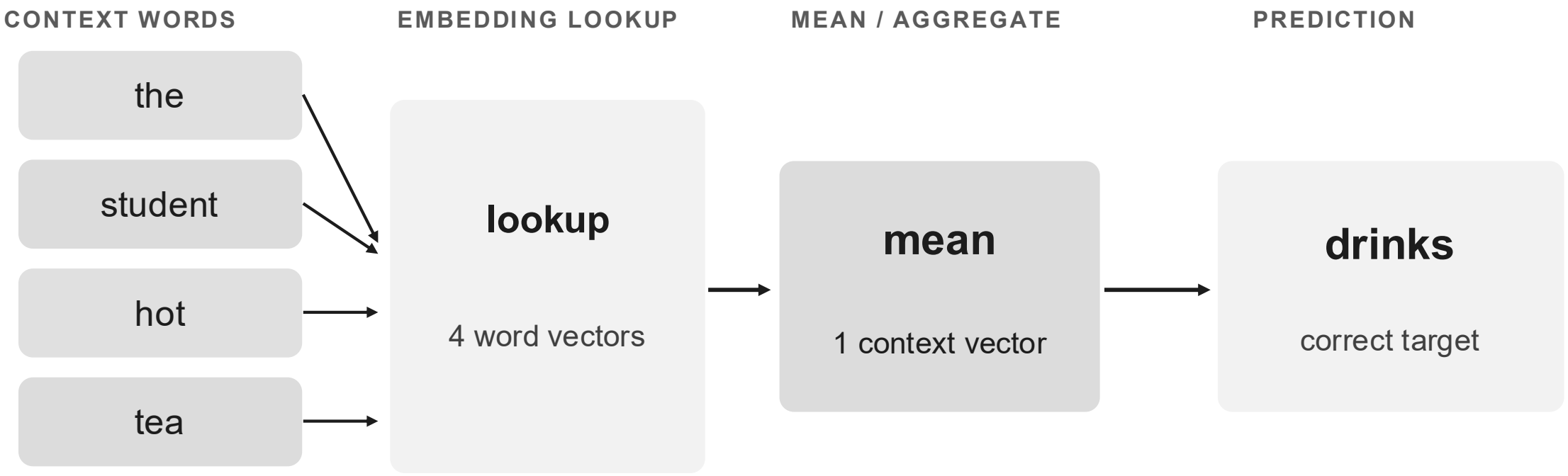
“the student _____ hot tea after class”



drinks

CBOW: context words → predicted center word

Four context words make one prediction



Many context vectors in → one word predicted out

How does CBOW combine context vectors?

Toy two-dimensional embeddings (not learned outputs)

the	(1.0, 0.0)
student	(0.8, 0.2)
hot	(0.2, 0.8)
tea	(0.4, 0.6)

Average each dimension

```
[(1+.8+.2+.4) / 4,
 (0+.2+.8+.6) / 4]
```

(0.6, 0.4)

The averaged vector feeds a prediction layer; averaging alone does not determine the predicted word.

How does CBOW improve with practice?

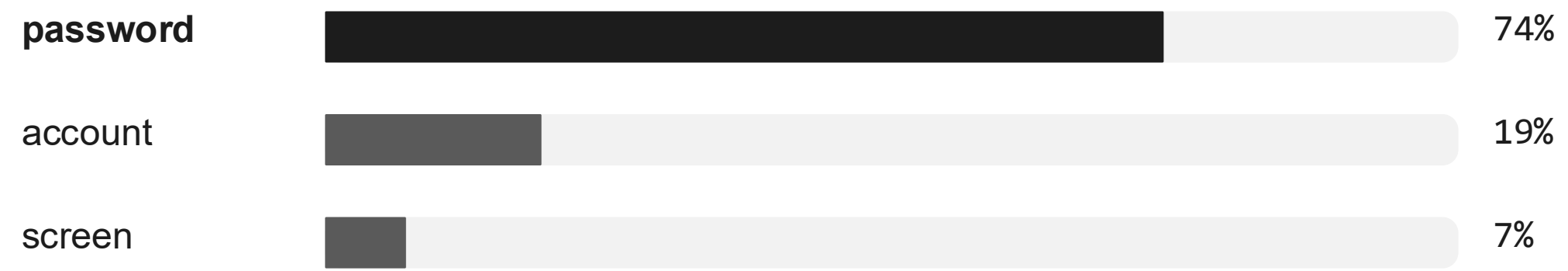
1	**Read context**	the · student · hot · tea
2	**Predict center**	model assigns scores to possible center words
3	**Measure error**	compare predicted scores with the target “drinks”
4	**Update vectors**	change trainable weights and repeat over corpus

Training data is self-supervised: the sentence supplies its own target.

An autocomplete-like “missing word” task

“Please reset my _____ before login.”

Possible prediction scores · illustrative only



The chart is synthetic and explains the prediction mechanism; it is not a trained Word2Vec result.

Skip-Gram = use one word to predict neighbors

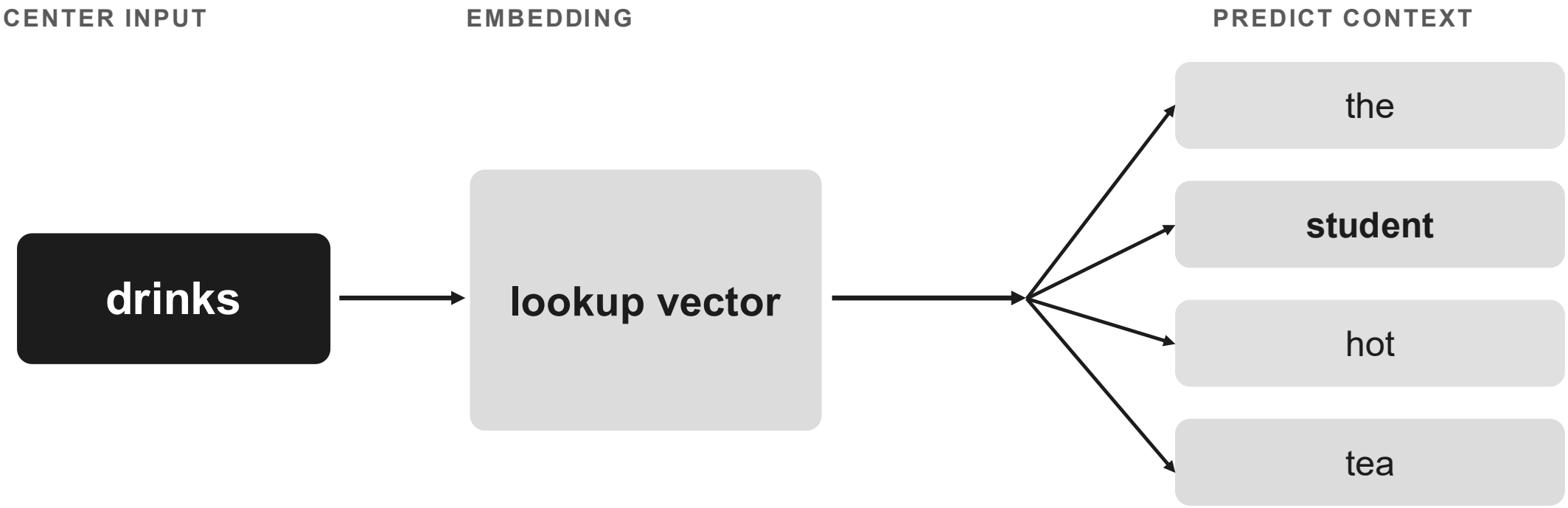
CENTER WORD: drinks



the **student** **hot** **tea**

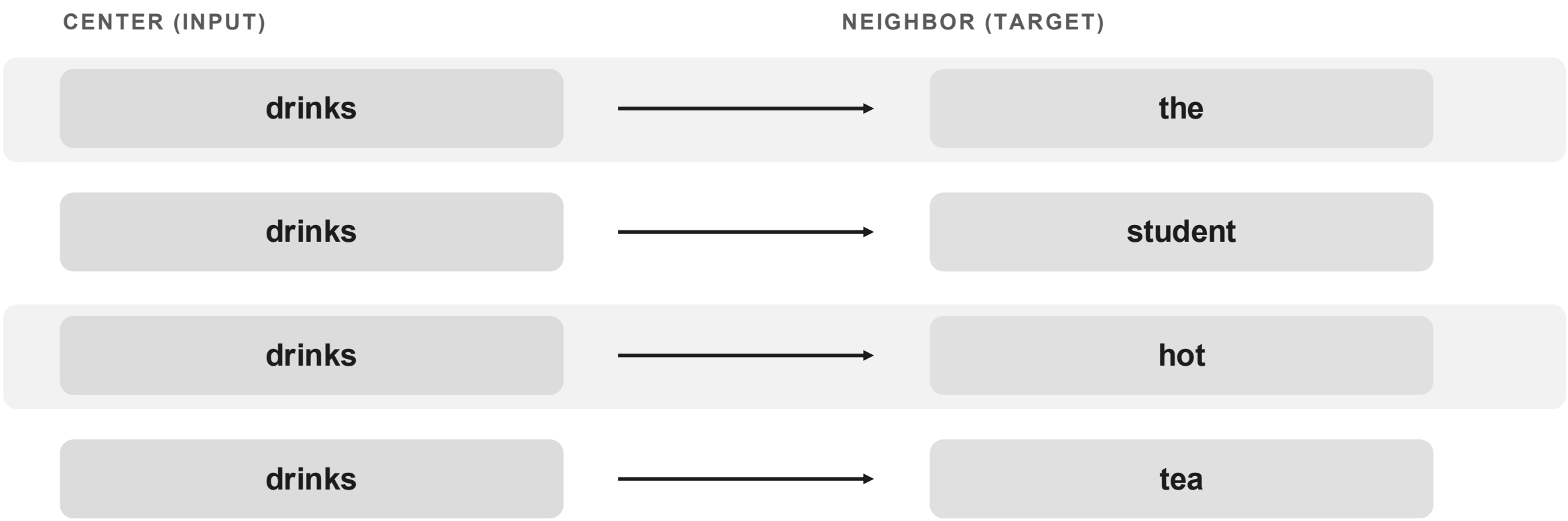
Skip-Gram: center word → each surrounding context word

One center word generates several predictions



One input → multiple center/context training pairs

Turn a window into four training pairs



Window size = 2 around “drinks”; the remaining words “after” and “class” fall outside the window.

The window moves across every sentence

center = student `the [student] drinks hot tea`

context: the · drinks · hot

center = drinks `the student [drinks] hot tea`

context: the · student · hot · tea

center = hot `student drinks [hot] tea after`

context: student · drinks · tea · after

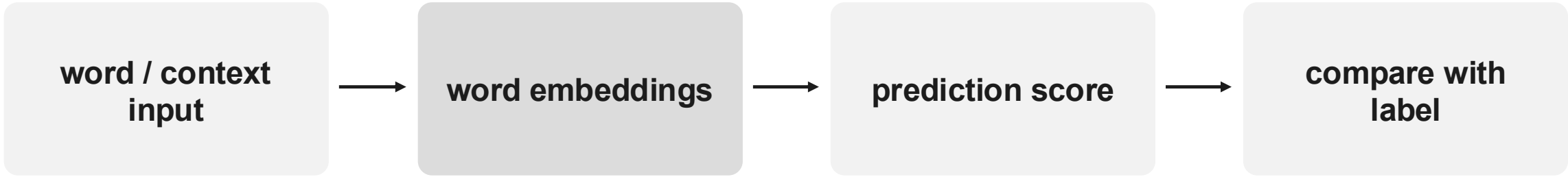
Teaching illustration using a fixed window of two tokens per side.

Negative sampling: learn from “not nearby” words



Conceptual noise examples—not guaranteed negative words in every corpus.

Prediction errors reshape word vectors



update the weights and repeat

Word2Vec’s main useful output is its learned vector table.

What is the difference?

PROPERTY	CBOW	SKIP-GRAM
Input	surrounding words	center word
Predict	one center word	neighbor words
Direction	many → one	one → many
Typical speed	often faster	often more training pairs
Less frequent words	may need care	often a useful choice

These are practical tendencies, not a universal accuracy ranking.

PRACTICAL DECISION

When might you choose each model?

CBOW: prioritize efficiency

Example: millions of common product-review sentences.

You want a quick, strong starting point for word vectors.

Try CBOW and validate on the downstream task.

Skip-Gram: focus on contexts

Example: specialized hospital notes with less frequent medical terms.

Skip-Gram may be worth testing for these rarer words.

Measure on your own data.

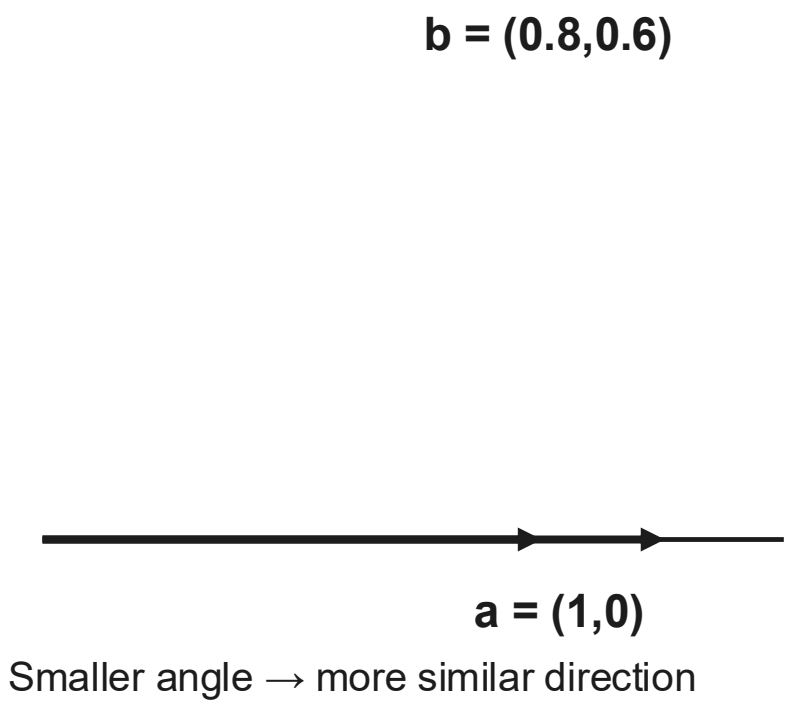
Always compare on the actual dataset and application.

Compare vectors with cosine similarity

Toy 2D example

```
a = (1.0, 0.0)
b = (0.8, 0.6)

cos(a,b) = 0.8
```



Numbers are worked examples, not genuine similarities from an external pretrained model.

INTERESTING PROPERTY

Word vectors can capture relationships

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Why this is useful

Some relationships emerge in the geometry of the learned vector space.

Important caution

Not guaranteed; vector analogies can reflect corpus bias or fail.

Historical Word2Vec analogy example from the original research literature.

APPLICATIONS

Where can static word embeddings help?

1

E-commerce search

“sneakers” ↔ “trainers”

Find alternative query terms

2

Support ticket analysis

“refund” · “return” · “exchange”

Group recurring ticket themes

3

Job matching

“developer” · “programmer”

Help match related skill terms

Illustrative use cases; actual systems may employ more recent embedding architectures.

Train CBOW and Skip-Gram in Gensim

```
from gensim.models import Word2Vec

sentences = [
  "students read books at school".split(),
  "teachers read books at school".split(),
  "students learn concepts in class".split(),
  "teachers explain concepts in class".split()
]

cbow = Word2Vec(sentences, vector_size=50, window=2,
                min_count=1, sg=0, epochs=100, workers=1)
skip = Word2Vec(sentences, vector_size=50, window=2,
                min_count=1, sg=1, epochs=100, workers=1)
```

Install once: pip install gensim | Gensim API: sg=0 (CBOW), sg=1 (Skip-Gram).

Inspect a word vector and nearby words

```
# Display five numbers from the word vector
print(cbow.wv["students"][:5])

# List the closest known words to “students”
print(cbow.wv.most_similar("students", topn=3))

# Compare directions learned by both algorithms
print(skip.wv.similarity("students", "teachers"))
```

Tiny corpus = code demonstration, NOT trustworthy semantics.

CLASS ACTIVITY · 4 MINUTES

You create the training examples

“the customer orders fresh pizza online today”

Center: orders

Window: 2 words each side

1. **Question 1** — What are the four context words?
2. **Question 2** — Write the single CBOW training example.
3. **Question 3** — Write all Skip-Gram training pairs.

Let us check the answers

CONTEXT WORDS

the · customer · fresh · pizza

CBOW

`[the, customer, fresh, pizza]`

→ **orders**

Skip-Gram

```
orders → the
orders → customer
orders → fresh
orders → pizza
```

Ask: “Which model is many-to-one?” Answer: CBOW.

Four misconceptions to avoid

1	**“CBOW uses word order.”**	Basic CBOW pools nearby vectors: word order is not modeled directly.
2	**“Skip = skip words.”**	Skip-Gram predicts nearby words from a center word.
3	**“One vector knows all meanings.”**	Static Word2Vec struggles with “bank” (river vs finance).
4	**“Tiny datasets are enough.”**	Good embeddings need substantial, representative text.

QUICK KNOWLEDGE CHECK

Three questions before we finish

1

Q1 · Many → one

Which architecture predicts a center word from its context?

2

Q2 · One → many

Which architecture predicts the center word's neighbors?

3

Q3 · Window = 2

How many neighboring positions can surround a middle token?

Answers: CBOW · Skip-Gram · four neighbors maximum

TAKEAWAYS

Word2Vec in three lines

1. **Word2Vec** — A family of methods to learn dense vectors from text.
2. **CBOW** — Context words → center word.
3. **Skip-Gram** — Center word → nearby context words.

Same local text; different prediction direction.

READ MORE

References and learning resources

Original Word2Vec paper

Mikolov, Chen, Corrado & Dean (2013)

https://arxiv.org/abs/1301.3781

Negative sampling + improvements

Mikolov, Sutskever, Chen, Corrado & Dean (2013)

https://arxiv.org/abs/1310.4546

Python reference

Gensim Word2Vec documentation

https://radimrehurek.com/gensim/models/word2vec.html